

# AWS Infrastructure

A highly available, fault-tolerant AWS architecture built entirely with Terraform to power advanced computer vision and 3D printing workflows.

---

## The Project Vision

---

This project demonstrates the deployment of a scalable AWS infrastructure using Terraform and Infrastructure as Code (IaC) principles.

The architecture integrates services such as VPC, subnets, NAT Gateway, security groups, Application Load Balancer, ECS, container registry, S3, and database resources, with the infrastructure fully defined and managed through Terraform.

## Cloud Architecture

---

## 1. The Network (VPC)

Deployed across two Availability Zones for true redundancy. Public subnets face the internet, while secure private subnets encapsulate all computational workloads to prevent direct access.

## 2. Traffic Routing

An Application Load Balancer (ALB) ingests user requests and routes them evenly. A NAT Gateway is utilized to allow private resources to fetch updates safely from the outside world.

## 3. Serverless Compute

AWS Fargate (ECS) serves as the container engine. It completely abstracts the underlying server maintenance, dynamically provisioning the exact CPU and RAM required to process the Python scripts.

## 4. Secure Storage

An Elastic Container Registry (ECR) securely stores the proprietary Docker images, while Amazon RDS (PostgreSQL) handles transactional database needs within a private subnet group.

# The Engineering Journey

---

Building this environment required solving real-world infrastructure challenges using Infrastructure as Code (IaC):

- ▶ **Overcoming the 503 Error:** Initial Fargate deployments failed because private containers could not reach Docker Hub. We successfully architected a NAT Gateway and properly routed the private subnet traffic to allow secure image pulls.
- ▶ **Port Mapping Alignment:** Resolved critical discrepancies between the Load Balancer Target Group ports and the container exposure ports, restoring full HTTP health checks.
- ▶ **Breaking Terraform State Locks:** When AWS blocked the deletion of a Target Group still attached to an ALB Listener, we implemented Terraform `lifecycle { create_before_destroy = true }` logic to seamlessly replace active resources without downtime.

## Terraform Implementation Example

---

The entire environment is version-controlled. Here is the lifecycle logic used to solve the Target Group dependency lock:

```
resource "aws_lb_target_group" "app" { name = "web-3d-tg-80" port = 80 protocol = "HTTP" vpc_id  
= aws_vpc.main.id target_type = "ip" lifecycle { create_before_destroy = true } }
```

Architected and Built by Rafik Gabriel Reis Ferreira

Contagem, Minas Gerais, Brazil